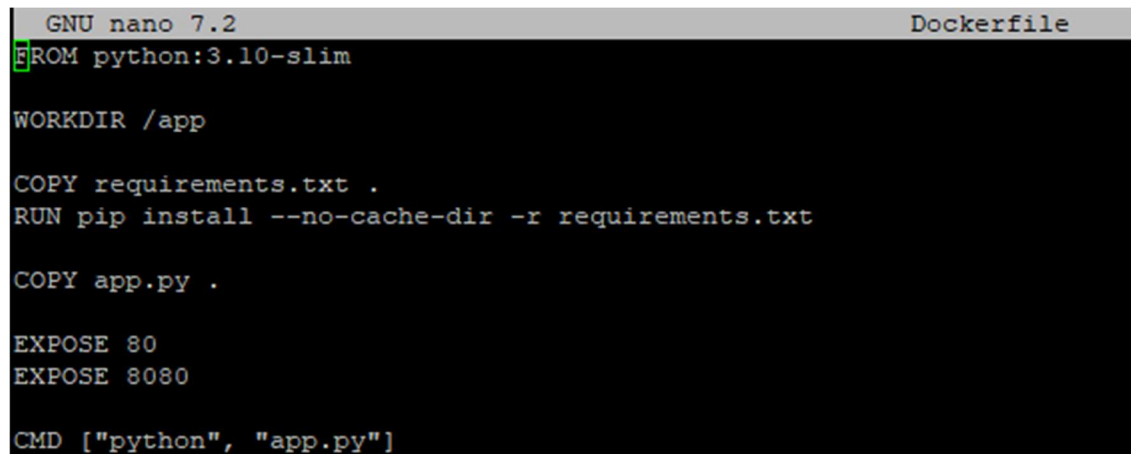


AIT labo 6 - Redondance applicative : k8s

Ce labo consiste en la mise en place d'une infrastructure kubernetes de 3 nœuds, afin de tester les mécanismes de redondance et de montée en charge (service de *disponibilité*).

1. Préparation d'un container de test.

1. Conception d'un container de test (par exemple sous Docker), permettant :
 - a. via le port de « contrôle » 8080, de définir le niveau de charge cpu (low, medium, high) demandé par le container.



```
GNU nano 7.2 Dockerfile
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 80
EXPOSE 8080

CMD ["python", "app.py"]
```

```

GNU nano 7.2 app.py
from flask import Flask, request
import socket
import threading
import time
import os

app = Flask(__name__)

LOAD_LEVEL = "low"
VERSION = "v2"

def cpu_load():
    global LOAD_LEVEL
    while True:
        if LOAD_LEVEL == "low":
            time.sleep(0.5)
        elif LOAD_LEVEL == "medium":
            for _ in range(1000000):
                pass
        elif LOAD_LEVEL == "high":
            while True:
                pass

threading.Thread(target=cpu_load, daemon=True).start()

@app.route("/", methods=["GET"])
def index():
    hostname = socket.gethostname()
    ip = socket.gethostbyname(hostname)
    return f"""
<html>
<body>
    <h1>Kubernetes Test Container</h1>
    <ul>
        <li><b>Hostname:</b> {hostname}</li>
        <li><b>IP:</b> {ip}</li>
        <li><b>CPU Load:</b> {LOAD_LEVEL}</li>
        <li><b>Version:</b> {VERSION}</li>
    </ul>
</body>
</html>
"""

@app.route("/load", methods=["GET"])
def set_load():
    global LOAD_LEVEL
    level = request.args.get("level")
    if level in ["low", "medium", "high"]:
        LOAD_LEVEL = level
        return f"CPU load set to {LOAD_LEVEL}\n"
    return "Invalid level (use low, medium or high)\n", 400

app.run(host="0.0.0.0", port=80)

```

- b. Retourne sur le port 80 (html) le hostname du container, son adresse IP, son niveau de charge (low, medium ou high) ainsi que la version (v1 ou v2)

Le container est implémenté en 2 versions (v1 et v2 ; même fonctions, mais retourne v1 ou v2).

Kubernetes Test Container

- **Hostname:** k8s-test-79d7dc49b5-gcb6g
- **IP:** 10.1.169.135
- **CPU Load:** low
- **Version:** v1

2. Déploiement d'une infrastructure k8s de test.

2. Installez la distribution microk8s, sur 3 VMs Ubuntu,

Configuration de chaque VM

- OS : Ubuntu Server 24.04.3 LTS
- CPU : 2 vCPU
- RAM : 2 Go
- Disque : 20 Go
- Réseau : NAT

```
sudo apt update && sudo apt upgrade -y
sudo snap install microk8s --classic
sudo usermod -aG microk8s $USER
newgrp microk8s
microk8s enable dns dashboard ingress metrics-server
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker $USER
newgrp docker
```

Il faut ensuite choisir un node et faire « microk8s add-node » pour les deux autres afin de les ajouter au cluster.

```

kevin@k8s-node1:~$ microk8s add-node
From the node you wish to join to this cluster, run the following:
microk8s join 192.168.88.137:25000/e4964f8988707d1eb9de8323368945e8/f5ed6f2311f0

Use the '--worker' flag to join a node as a worker not running the control plane, eg:
microk8s join 192.168.88.137:25000/e4964f8988707d1eb9de8323368945e8/f5ed6f2311f0 --worker

If the node you are adding is not reachable through the default interface you can use one of the following:
microk8s join 192.168.88.137:25000/e4964f8988707d1eb9de8323368945e8/f5ed6f2311f0
kevin@k8s-node1:~$
kevin@k8s-node2:~$ microk8s join 192.168.88.137:25000/e4964f8988707d1eb9de8323368945e8/f5ed6f2311f0
[sudo] password for kevin:
Contacting cluster at 192.168.88.137
Waiting for this node to finish joining the cluster. . . . .
Successfully joined the cluster.
kevin@k8s-node2:~$

```

```

kevin@k8s-node1:~$ microk8s kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-node1     Ready     <none>   43m   v1.32.9
k8s-node2     Ready     <none>   22m   v1.32.9
k8s-node3     Ready     <none>   2s    v1.32.9
kevin@k8s-node1:~$

```

3. Installez l'add-on `metallb` comme *load balancer* externe.

Cette manipulation se fait sur un seul node car c'est le cluster Kubernetes entier qui est modifier

```

kevin@k8s-node1:~$ microk8s enable metallb
Infer repository core for add-on metallb
Enabling Metallb
Enter each IP address range delimited by comma (e.g. '10.64.140.43-10.64.140.49,192.168.0.105-192.168.0.111'): 192.168.80.240-192.168.80.250
Applying Metallb manifest
customresourcedefinition.apiextensions.k8s.io/addresspools.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bfdprofiles.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgpadvertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgppeers.metallb.io created
customresourcedefinition.apiextensions.k8s.io/communities.metallb.io created
customresourcedefinition.apiextensions.k8s.io/ipaddresspools.metallb.io created
customresourcedefinition.apiextensions.k8s.io/l2advertisements.metallb.io created
namespace/metallb-system created
serviceaccount/controller created
serviceaccount/speaker created
clusterrole.rbac.authorization.k8s.io/metallb-system:controller created
clusterrole.rbac.authorization.k8s.io/metallb-system:speaker created
role.rbac.authorization.k8s.io/controller created
role.rbac.authorization.k8s.io/pod-lister created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:controller created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:speaker created
rolebinding.rbac.authorization.k8s.io/controller created
secret/webhook-server-cert created
service/webhook-service created
rolebinding.rbac.authorization.k8s.io/pod-lister created
daemonset.apps/speaker created
deployment.apps/controller created
validatingwebhookconfiguration.admissionregistration.k8s.io/validating-webhook-configuration created
Waiting for Metallb controller to be ready.
error: timed out waiting for the condition on deployments/controller
Metallb controller is still not ready
deployment.apps/controller condition met
ipaddresspool.metallb.io/default-addresspool created
l2advertisement.metallb.io/default-advertise-all-pools created
Metallb is enabled

```

3. Test de la redondance de l'infrastructure

Votre container sera déployé sur votre infrastructure k8s, en utilisant un *service*.

- En utilisant la fonction de *deployment*, déployez la version v1 de votre container, afin que 3 *pods* tournent en permanence.

```
GNU nano 7.2 deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: k8s-test
spec:
  replicas: 3
  selector:
    matchLabels:
      app: k8s-test
  template:
    metadata:
      labels:
        app: k8s-test
    spec:
      containers:
      - name: k8s-test
        image: localhost:32000/k8s-test:v1
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 80
```

```
kevin@k8s-node1:~/k8s-test$ microk8s kubectl apply -f deployment.yaml
deployment.apps/k8s-test created
kevin@k8s-node1:~/k8s-test$ microk8s kubectl get deploy
microk8s kubectl get pods -o wide
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
k8s-test      3/3     3            3           47s
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
k8s-test-79d7dc49b5-gcb6g  1/1     Running    0          48s   10.1.169.135  k8s-node2    <none>           <none>
k8s-test-79d7dc49b5-hsj24  1/1     Running    0          48s   10.1.107.197  k8s-node3    <none>           <none>
k8s-test-79d7dc49b5-x8rhw  1/1     Running    0          48s   10.1.36.71    k8s-node1    <none>           <none>
```

- Testez la répartition de charge en accédant au service (port 80 des containers) via le load balancer externe.

```
GNU nano 7.2 service.yaml
apiVersion: v1
kind: Service
metadata:
  name: k8s-test-svc
spec:
  type: LoadBalancer
  selector:
    app: k8s-test
  ports:
  - port: 80
    targetPort: 80
```

```

kevin@k8s-nodel:~/k8s-test$ microk8s kubectl apply -f service.yaml
service/k8s-test-svc created
kevin@k8s-nodel:~/k8s-test$ microk8s kubectl get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
k8s-test      3/3     3            3           2m42s
kevin@k8s-nodel:~/k8s-test$ microk8s kubectl get svc -o wide
NAME          TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE   SELECTOR
k8s-test-svc  LoadBalancer 10.152.183.179 192.168.186.240 80:32297/TCP     9s    app=k8s-test
kubernetes    ClusterIP      10.152.183.1  <none>        443/TCP          37m   <none>

```

Kubernetes Test Container

- **Hostname:** k8s-test-79d7dc49b5-hsj24
- **IP:** 10.1.107.197
- **CPU Load:** low
- **Version:** v1

Kubernetes Test Container

- **Hostname:** k8s-test-79d7dc49b5-x8rhw
- **IP:** 10.1.36.71
- **CPU Load:** low
- **Version:** v1

Kubernetes Test Container

- **Hostname:** k8s-test-79d7dc49b5-gcb6g
- **IP:** 10.1.169.135
- **CPU Load:** low
- **Version:** v1

6. Terminez une instance de votre container/pods et vérifiez qu'une instance supplémentaire est créée.

```

kevin@k8s-nodel:~/k8s-test$ microk8s kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
k8s-test-79d7dc49b5-gcb6g  1/1     Running   0          4h53m  10.1.169.135  k8s-node2  <none>           <none>
k8s-test-79d7dc49b5-hsj24  1/1     Running   0          4h53m  10.1.107.197  k8s-node3  <none>           <none>
k8s-test-79d7dc49b5-x8rhw  1/1     Running   0          4h53m  10.1.36.71    k8s-nodel  <none>           <none>
kevin@k8s-nodel:~/k8s-test$ microk8s kubectl delete pod k8s-test-79d7dc49b5-x8rhw
pod "k8s-test-79d7dc49b5-x8rhw" deleted

kevin@k8s-nodel:~/k8s-test$
kevin@k8s-nodel:~/k8s-test$ microk8s kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
k8s-test-79d7dc49b5-gcb6g  1/1     Running   0          4h54m  10.1.169.135  k8s-node2  <none>           <none>
k8s-test-79d7dc49b5-h8mg8  1/1     Running   0          37s    10.1.107.197  k8s-node3  <none>           <none>
k8s-test-79d7dc49b5-hsj24  1/1     Running   0          4h54m  10.1.107.197  k8s-node3  <none>           <none>

```

7. Arrêtez un nœud microk8s et vérifiez l'accès à vos containers.

```

kevin@k8s-nodel:~/k8s-test$ microk8s kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-nodel     Ready     <none>   5h46m  v1.32.9
k8s-node2    NotReady  <none>   5h42m  v1.32.9
k8s-node3    Ready     <none>   5h41m  v1.32.9
kevin@k8s-nodel:~/k8s-test$ microk8s kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
k8s-test-79d7dc49b5-gcb6g  1/1     Terminating   0      5h11m  10.1.169.135  k8s-node2  <none>           <none>
k8s-test-79d7dc49b5-h8mg8  1/1     Running        0      17m    10.1.36.72    k8s-nodel  <none>           <none>
k8s-test-79d7dc49b5-hsj24  1/1     Running        0      5h11m  10.1.107.197  k8s-node3  <none>           <none>
k8s-test-79d7dc49b5-nfhrv  1/1     Running        0      8m2s   10.1.107.199  k8s-node3  <none>           <none>

```


4. Montée en charge

8. Testez la fonction de montée en charge et d'augmentation du nombre de containers/pods.

```

kevin@k8s-node1:~/k8s-test$ microk8s kubectl get deployment k8s-test
microk8s kubectl get pods
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
k8s-test      3/3     3            3           5h16m
NAME          READY   STATUS    RESTARTS   AGE
k8s-test-79d7dc49b5-h8mg8  1/1     Running   0          22m
k8s-test-79d7dc49b5-hsj24  1/1     Running   0          5h16m
k8s-test-79d7dc49b5-nfhrv  1/1     Running   0          12m
kevin@k8s-node1:~/k8s-test$ microk8s kubectl scale deployment k8s-test --replicas=6
deployment.apps/k8s-test scaled
kevin@k8s-node1:~/k8s-test$ microk8s kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
k8s-test-79d7dc49b5-6216l  1/1     Running   0          6s    10.1.169.139  k8s-node2    <none>           <none>
k8s-test-79d7dc49b5-8fv7g  1/1     Running   0          6s    10.1.169.138  k8s-node2    <none>           <none>
k8s-test-79d7dc49b5-h8mg8  1/1     Running   0          22m   10.1.36.72    k8s-node1    <none>           <none>
k8s-test-79d7dc49b5-hsj24  1/1     Running   0          5h16m 10.1.107.197  k8s-node3    <none>           <none>
k8s-test-79d7dc49b5-ldlnx  1/1     Running   0          6s    10.1.36.73    k8s-node1    <none>           <none>
k8s-test-79d7dc49b5-nfhrv  1/1     Running   0          13m   10.1.107.199  k8s-node3    <none>           <none>

```

5. Montée en charge

9. Testez la fonction de migration (rolling update) d la version v1 à la version v2 de votre container.

```

kevin@k8s-node1:~/k8s-test$ microk8s kubectl set image deployment/k8s-test \
  k8s-test=localhost:32000/k8s-test:v2
deployment.apps/k8s-test image updated
kevin@k8s-node1:~/k8s-test$ microk8s kubectl rollout status deployment/k8s-test
Waiting for deployment "k8s-test" rollout to finish: 5 of 6 updated replicas are available...
deployment "k8s-test" successfully rolled out
kevin@k8s-node1:~/k8s-test$ microk8s kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
k8s-test-76bfcf8bbb-5n9n4  1/1     Running   0          26s   10.1.169.141  k8s-node2    <none>           <none>
k8s-test-76bfcf8bbb-bl942  1/1     Running   0          26s   10.1.107.201  k8s-node3    <none>           <none>
k8s-test-76bfcf8bbb-fd46x  1/1     Running   0          29s   10.1.107.200  k8s-node3    <none>           <none>
k8s-test-76bfcf8bbb-lpp7v  1/1     Running   0          25s   10.1.36.75    k8s-node1    <none>           <none>
k8s-test-76bfcf8bbb-qjrcg  1/1     Running   0          29s   10.1.169.140  k8s-node2    <none>           <none>
k8s-test-76bfcf8bbb-t251v  1/1     Running   0          29s   10.1.36.74    k8s-node1    <none>           <none>
k8s-test-79d7dc49b5-6216l  1/1     Terminating 0       110s  10.1.169.139  k8s-node2    <none>           <none>
k8s-test-79d7dc49b5-8fv7g  1/1     Terminating 0       110s  10.1.169.138  k8s-node2    <none>           <none>
k8s-test-79d7dc49b5-h8mg8  1/1     Terminating 0       24m   10.1.36.72    k8s-node1    <none>           <none>
k8s-test-79d7dc49b5-hsj24  1/1     Terminating 0       5h18m 10.1.107.197  k8s-node3    <none>           <none>
k8s-test-79d7dc49b5-ldlnx  1/1     Terminating 0       110s  10.1.36.73    k8s-node1    <none>           <none>
k8s-test-79d7dc49b5-nfhrv  1/1     Terminating 0       14m   10.1.107.199  k8s-node3    <none>           <none>

```